

Iterative Methods for Unconstrained Continuous Optimization

Design and Analysis of Algorithms

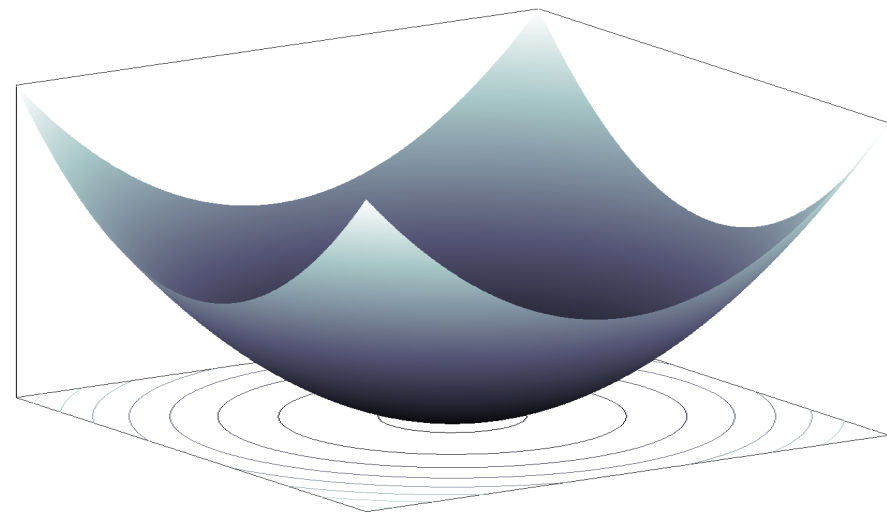
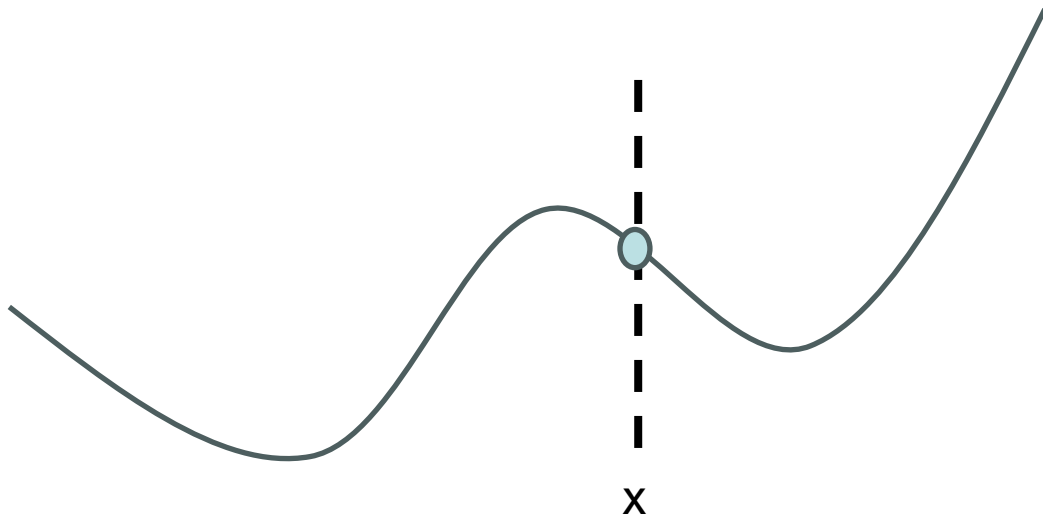
Brian C. Dean

**School of Computing
Clemson University**



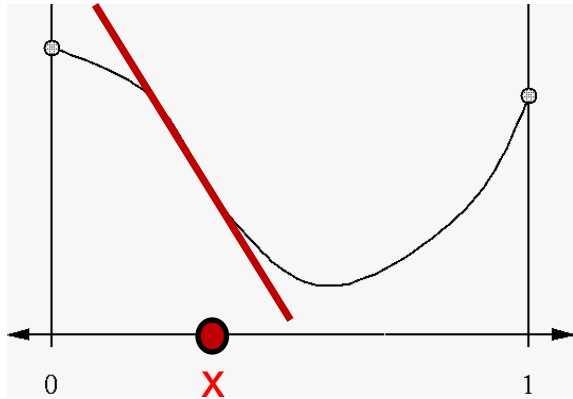
Iterative Refinement

- Key questions: how to choose direction and step size in each iteration?
- Typical tradeoff between:
 - Many low-quality steps (e.g., stochastic gradient descent) and
 - Fewer but higher quality steps (e.g., Newton's method)



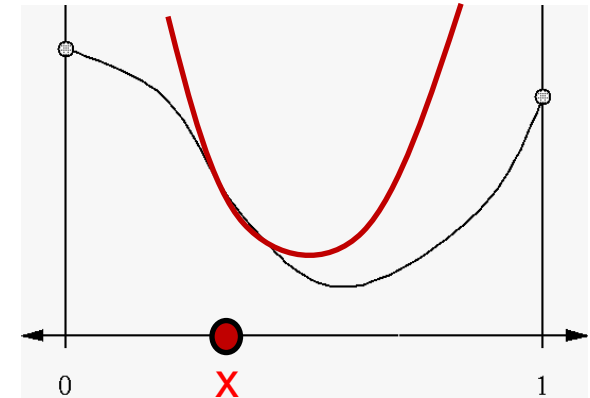
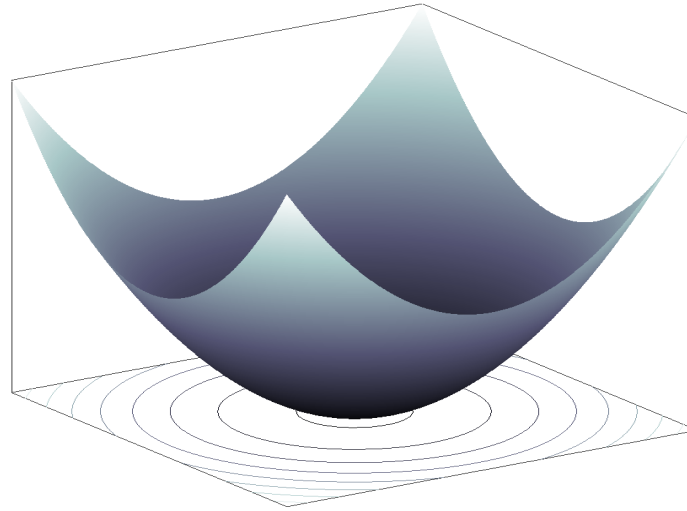
Can You Find Derivatives?

- If no derivatives available, we often fall back to Nelder Mead or population-based approaches.



Gradient Descent: use first derivative to locally approximate function around x as a line / hyperplane.

$$f(x + y) \approx f(x) + y f'(x)$$

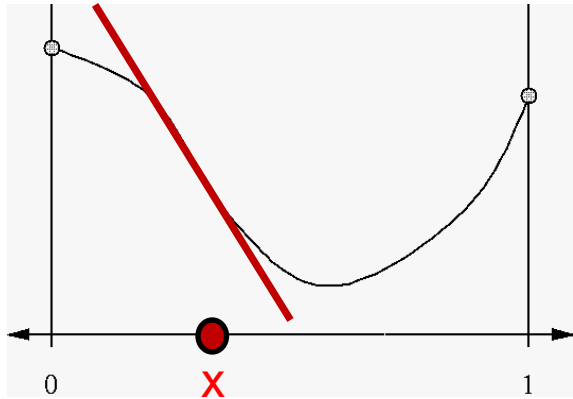


Newton's Method Use second derivative to approximate function around x locally like a quadratic.

$$f(x + y) \approx f(x) + y f'(x) + y^2 \frac{f''(x)}{2}$$

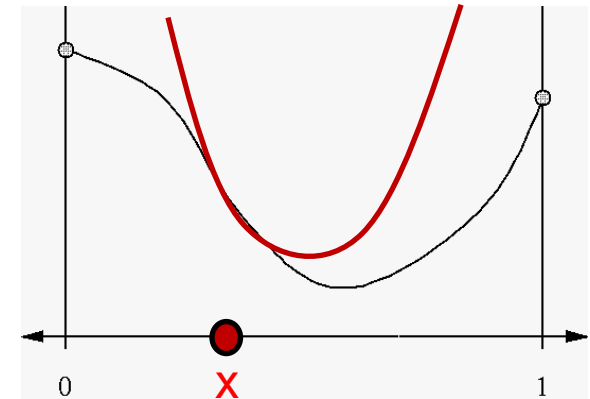
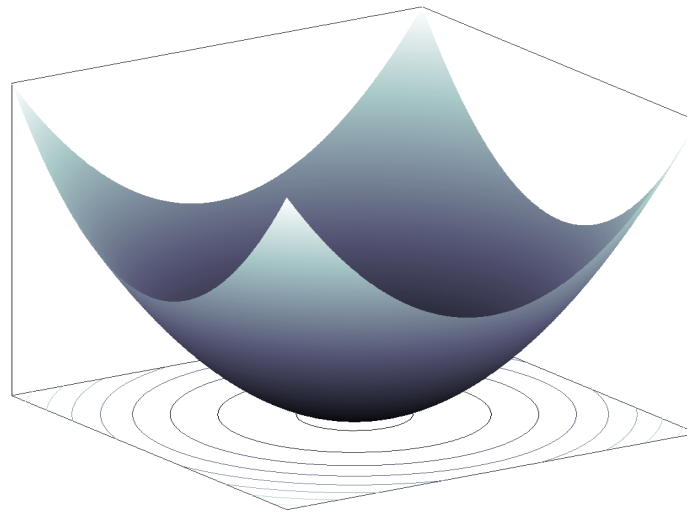
Can You Find Derivatives?

- If no derivatives available, we often fall back to Nelder Mead or population-based approaches.



Gradient Descent: use first derivative to locally approximate function around x as a line / hyperplane.

$$f(x + y) \approx f(x) + y^T \nabla f(x)$$

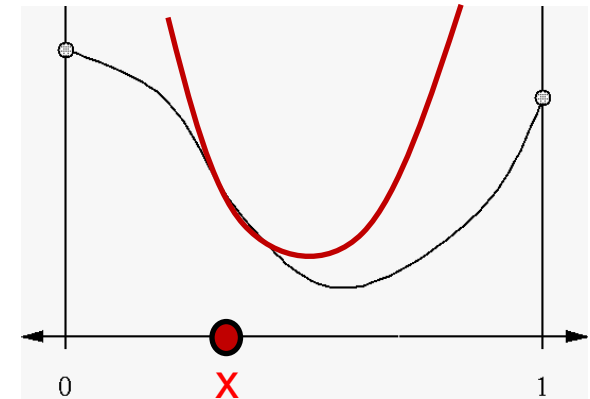


Newton's Method Use second derivative to approximate function around x locally like a quadratic.

$$f(x + y) \approx f(x) + y^T \nabla f(x) + y^T \frac{\nabla^2 f(x)}{2} y$$

Newton's Method

- Fewer “high quality” steps.
- Bad news: In d dimensions, each step requires solving a linear system with d variables.
- Good news: near optimum solution, when well-approximated by a quadratic, we get very fast “quadratic convergence”.
- *Levenberg-Maquardt* method: use weighted average of gradient and Newton directions (shifting more to Newton over time).



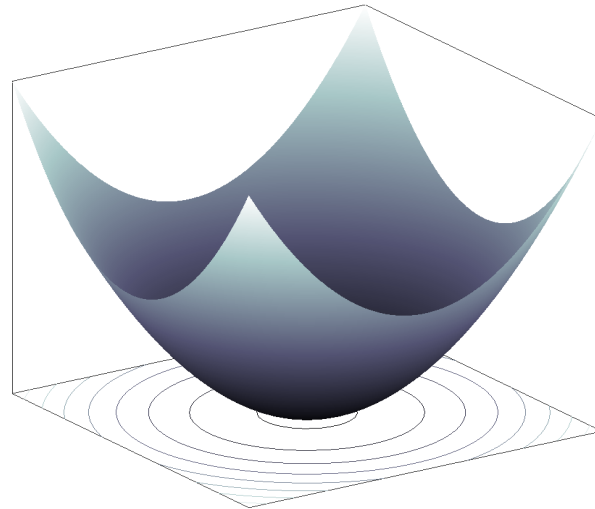
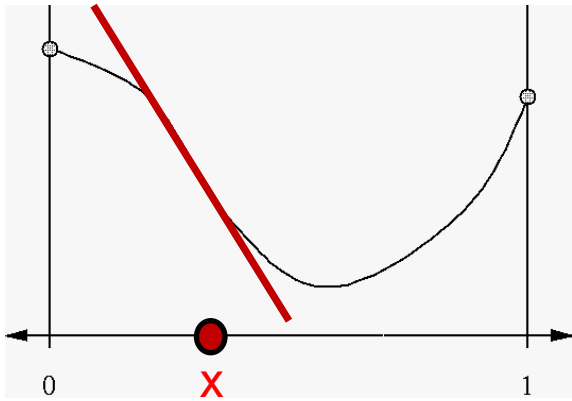
Newton's Method Use second derivative to approximate function around x locally like a quadratic.

$$f(x + y) \approx f(x) + y^T \nabla f(x) + y^T \frac{\nabla^2 f(x)}{2} y$$

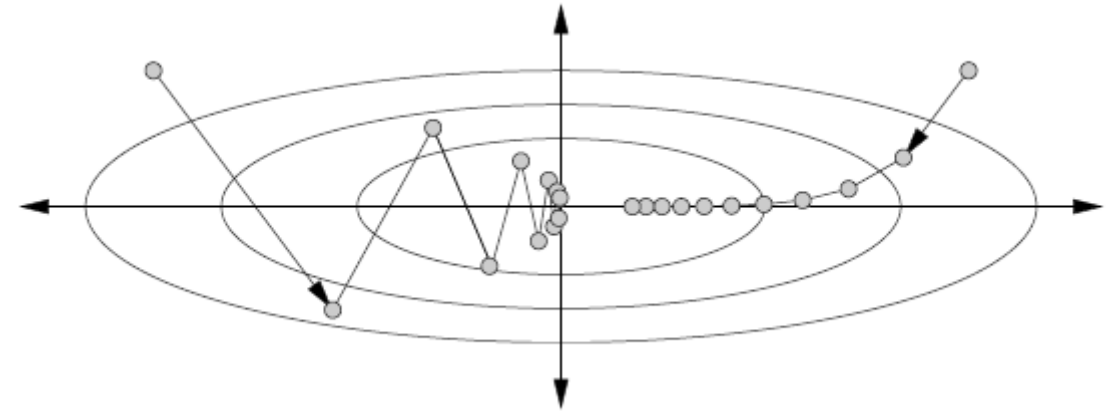
Gradient Descent

- The gradient vector $\nabla f(x)$ points in the direction in which f increases the fastest at x , so to minimize f , move some amount in $-\nabla f(x)$ direction:

$$x' = x - \alpha \nabla f(x)$$



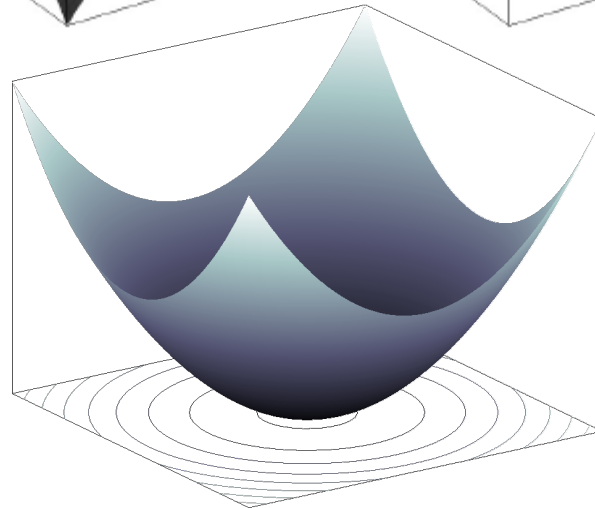
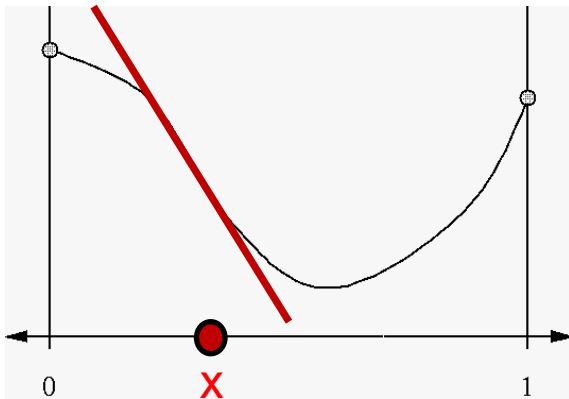
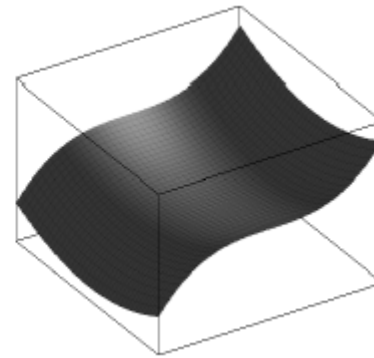
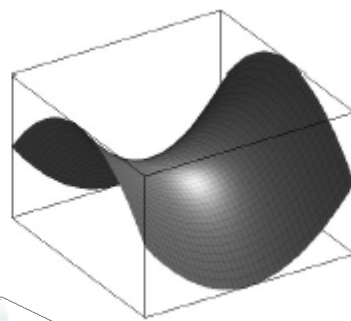
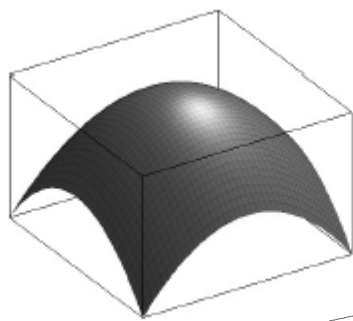
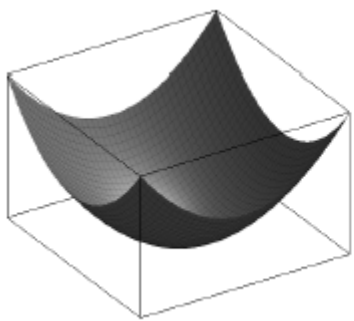
$K = 1$



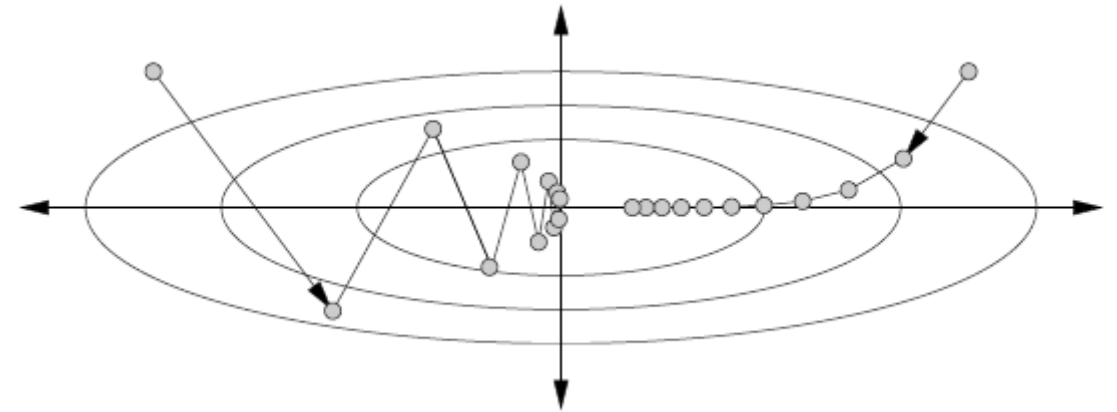
$K \gg 1$

Gradient Descent

- $\nabla f(x) = 0$ is a “stationary point” in general, or a minimum if f is convex.



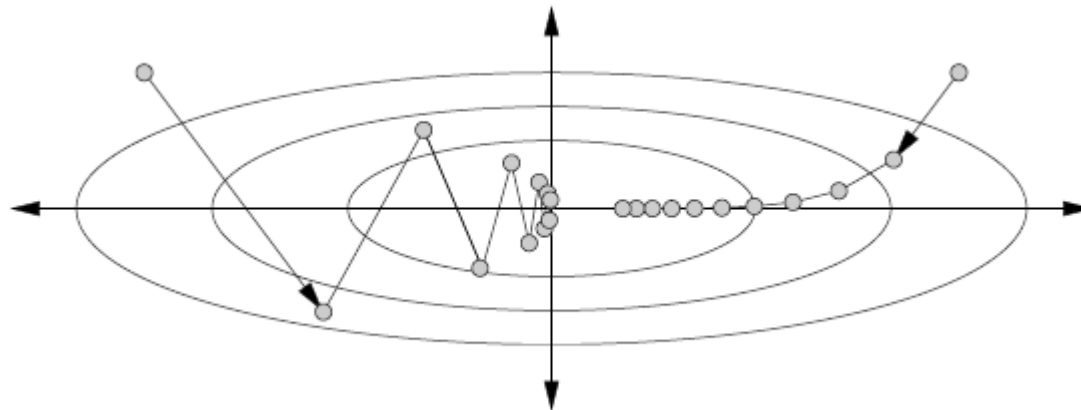
$K = 1$



$K \gg 1$

Step Size Calculation

- Two prominent options:
 - **Fixed:** ideally $\alpha = 1 / M$ (or c / M where $0 < c \leq 1$), where M (measuring “curvature rate”) is an upper bound on f'' or the largest eigenvalue of $\nabla^2 f$. Note: Smaller steps as $\nabla f \rightarrow 0$.
 - **Adaptive:** often use 1d backtracking line search to find a step size satisfying the “Armijo rule”, giving a decrease per step proportional to what gradient magnitude $\|\nabla f(x)\|$ suggests should be possible.



Convergence Rates

- Assume using fixed or adaptive step size as just discussed.
- If f is strongly convex ($f'' > 0$)
 - Function error at time t , $\text{err}(t) = f(x_t) - f(x^*)$ decreases at an *exponential* rate: $\text{err}(t) \leq e^{-\Theta(t/\kappa)} \text{err}(0)$. (“linear” convergence)
 - Equivalently: achieving $\text{err}(t) / \text{err}(0) \leq \varepsilon$ needs $O(\kappa \log(1/\varepsilon))$ steps.
- If f only convex ($f'' \geq 0$)
 - Weaker bounds: $\text{err}(t) = O(1/t)$ dM, with d being squared distance of initial guess x from x^* . (rate usually described as just $O(1/t)$).

Accelerated Gradient Descent

- “Heavy ball” method adds “momentum”:

$$x' = x - \alpha \nabla f(x) + \beta(\Delta x),$$

where Δx is a vector describing the last step we took.

- Nesterov’s method (safer): $x' = x - \alpha(\nabla f(x) + \beta(\Delta x)) + \beta(\Delta x)$
- If f is strongly convex ($f'' > 0$)
 - To get $\text{err}(t) / \text{err}(0) \leq \varepsilon$: $O(\kappa \log(1/\varepsilon))$ steps $\rightarrow O(\sqrt{\kappa} \log(1/\varepsilon))$
- If f only convex ($f'' \geq 0$)
 - Error rate $O(1 / t) \rightarrow O(1 / t^2)$
- Setting parameters α and β requires care to make this work.

Stochastic Gradient Descent

- Model fitting often involves an error / loss function like $f(x) = \sum f_i(x)$, summed over per-data-point errors.
- $\nabla f(x) = \sum \nabla f_i(x)$ straightforward but expensive to compute.
- Update x according to suggestion of just one data point:
$$x' = x - \alpha \nabla f_i(x) \quad (\text{note: } \mathbf{E}[\nabla f_i(x)] = \nabla f(x))$$
- Also works if you use noisy estimates for $\nabla f(x)$.
- Best convergence rates for strongly convex and convex cases are $O(1 / t)$ and $O(1 / \sqrt{t})$ with appropriate tuning.
 - Note: same for “subgradient” (stochastic) gradient descent.

Final Ideas

- Coordinate descent (“alternating iterative refinement”). Sometimes in blocks.
- Conjugate gradient method (discussed later).
- “Quasi”-Newton methods
- “Simulated annealing” / adding randomness to gradient descent (discussed later).